Stack technique et code minimal

Plateforme de découverte — restaurants et meublés à Douala

Date : 5 octobre 2026

Contenu : choix de la stack, puis le code minimal de chaque pièce, avec le rôle de chaque fichier et les commandes pour lancer l'application.

État : le code est écrit et vérifié. Les deux projets compilent sans erreur TypeScript.

Dossier : `/home/junior_dev/plateforme`

Sommaire

1. **Partie 1 — La stack, et pourquoi**
2. **Partie 2 — La base de données** (fichier 1)
3. **Partie 3 — Le back-end** (fichiers 2 à 4)
4. **Partie 4 — Le front-end** (fichiers 5 à 7)
5. **Partie 5 — Montage pas à pas**
6. **Partie 6 — Vérifier que ça marche**

PARTIE 1 — LA STACK, ET POURQUOI

1. La décision de fond : un seul langage

TypeScript partout. Un seul langage pour le tracker, l'API, le site public et le tableau de bord. Une seule base de code, une seule compétence à recruter, un seul environnement.

Ce n'est pas une préférence de goût. C'est une question de friction : dans un projet naissant, chaque langage supplémentaire est un déploiement supplémentaire, un recruiting supplémentaire, et un endroit de plus où une erreur se cache.

Couche	Choix	Pourquoi celui-là
Langage	**TypeScript** sur Node.js 22	Les SDK officiels Meta et WhatsApp sont Node-first. Le mobile money est du HTTP et du JSON. Un seul langage pour toute l'équipe.
Back-end	**Fastify** + Zod	Léger, rapide, fortement typé. La réception de webhooks est son cas d'usage principal.
Front public	**Next.js** (App Router, rendu serveur)	Le SEO est la trajectoire T1 du plan de trafic. Il faut du HTML rendu côté serveur, pas du JavaScript client.
Tableau de bord	React, dans le même projet	Un seul build, un seul déploiement à surveiller.
Base	**PostgreSQL**	Une seule base pour tout. JSONB pour les charges utiles, vraies contraintes, et une vue SQL qui fait le travail d'agrégation à la place du code.
Cache	**Redis**	Limitation de débit, sessions, compteurs. À ajouter au palier 1.
Files d'attente	BullMQ sur Redis	Obligatoire dès que WhatsApp entre en jeu : découpler une action lente d'une requête utilisateur.
Photos	Cloudflare R2 + `sharp`	Redimensionnement et WebP à la volée. Important quand la data coûte environ 1 000 FCFA le gigaoctet.
Observabilité	Sentry + PostHog auto-hébergé	PostHog en local : les données de tes utilisateurs ne quittent pas le pays.
Infra	Ubuntu LTS, Caddy, systemd	Sauvegarde `pg_dump` quotidienne, et surtout **restaurée et testée**.

2. Ce que je n'utiliserais pas

Écarté	Raison
PHP / Laravel	Excellent, mais ici tout le métier est du tracking asynchrone temps réel, et les SDK Meta sont Node-first. Si l'équipe est experte Laravel, ça se défend. Sinon, deux langages à maintenir.
Python / FastAPI	Très bon, mais moins bon sur WhatsApp et sur le front. À réintroduire plus tard si tu fais de la recommandation ou de l'analyse comportementale.
Go	Techniquement supérieur pour du trafic de clics. Mais tu gères quelques centaines de milliers de clics par jour au début, pas des millions. Go coûte du temps d'itération et un bassin de recrutement plus étroit.
MongoDB	Aucune raison de l'utiliser ici. PostgreSQL fait le travail et évite une deuxième base à opérer.
Next.js seul, sans back-end séparé	Acceptable au démarrage. Mais le tracker doit vivre dans son propre service dès que le trafic monte : il est sensible à la latence et ne doit pas partager un runtime avec le catalogue.

3. Les compétences qui ne périment pas

C'est le vrai critère de durabilité, davantage que la technologie :

- **SQL**. Aucun framework ne le remplace, et tes données d'attribution sont fondamentalement relationnelles : un clic appartient à un établissement, à une date.

- **La sémantique HTTP**. Codes de statut, redirections, idempotence. Ta redirection serveur *est* ton cœur de métier.
- **L'idée de file d'attente**. Découpler une action lente d'une requête utilisateur.

> Une règle à retenir : **n'ajoute une technologie que lorsqu'elle te débloque un problème que tu as réellement rencontré.**

4. L'arborescence

```
plateforme/
├── demarrer.sh                lance et arrete tout (voir section 5)
├── docker-compose.yml         la base Postgres, avec le fuseau Africa/Douala
├── db/
│   └── schema.sql             FICHIER 1 — offre, clics, conversations, vue
├── backend/
│   ├── package.json
│   ├── tsconfig.json
│   └── src/
│       ├── db.ts              FICHIER 2 — la preuve du clic
│       ├── server.ts          FICHIER 3 — les routes HTTP
│       └── seed.ts            FICHIER 4 — données de départ
└── frontend/
    ├── package.json
    ├── tsconfig.json
    └── app/
        ├── layout.tsx         FICHIER 5 — racine du site
        ├── page.tsx           FICHIER 6 — le catalogue
        ├── e/[slug]/page.tsx FICHIER 7 — la fiche + le bouton
        └── dashboard/page.tsx FICHIER 8 — la chaîne de mesure
```

Le code complet des huit fichiers suit. Chaque fichier est commenté sur son rôle, et le PDF est conçu pour être lu à côté de l'éditeur.

PARTIE 2 — LA BASE DE DONNÉES

`db/schema.sql`
Rôle : définir les trois tables du projet et la vue qui produit le tableau de bord. C'est le seul endroit où vit la règle anti-fraude.

Trois idées structurent ce fichier :

1. **etablissements** — l'offre, avec le vrai numéro WhatsApp du commerce.
2. **clics** — la preuve, horodatée, écrite par le serveur uniquement.
3. **conversations** — ce que le client déclare, filtré par code de référence.

Il n'y a pas de table `reservations`. On ne touche pas aux transactions : c'est ce qui élimine tout le risque financier du modèle.

> **Le point le plus important du fichier** est l'index unique `idx_clics_anti_fraude` à la fin. Il interdit deux clics comptés pour le même établissement, le même visiteur, le même jour. Ce n'est pas du code applicatif qu'on peut contourner : c'est la base elle-même qui refuse la seconde ligne.

```
-- Schema de la base. C'est ici que tout se joue : chaque clic est un fait
-- enregistre cote serveur, jamais declare par un client.

-- 1. L'OFFRE ------------------------------------------------------------
```

```
CREATE TABLE etablissements (
  id            SERIAL PRIMARY KEY,
  slug          TEXT UNIQUE NOT NULL,
  nom           TEXT NOT NULL,
  categorie     TEXT NOT NULL CHECK (categorie IN ('restaurant', 'meuble')),
  quartier      TEXT NOT NULL,
  prix_affiche  INT NOT NULL,                  -- en FCFA, affiche AVANT le clic
  devisee       TEXT NOT NULL,                 -- "Plat du jour", "par nuit"
  photo_url     TEXT,
  whatsapp      TEXT NOT NULL,                 -- numero du commerce
  ref_code      TEXT UNIQUE NOT NULL,          -- EST-042-A7, filtre anti-declaration
  actif         BOOLEAN NOT NULL DEFAULT true, -- l'abonnement : visible ou non
  cree_le       TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_etablissements_categorie
  ON etablissements (categorie, quartier) WHERE actif;

-- 2. LA PREUVE DU CLIC ------------------------------------------------
-- Cette table est le coeur technique du projet.
-- Elle est ecrite par le serveur, jamais par le client. Un clic est un fait.

CREATE TABLE clics (
  id            BIGSERIAL PRIMARY KEY,
  etablissement_id INT NOT NULL REFERENCES etablissements(id),
  token         TEXT UNIQUE NOT NULL,          -- voyage dans le lien et dans wa.me
  ip_hash       TEXT NOT NULL,                 -- jamais l'IP en clair : hachee avec du sel
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  compte        BOOLEAN NOT NULL DEFAULT true, -- false si doublon detecte
  motif_exclu   TEXT                           -- NULL si compte, sinon le motif
);

CREATE INDEX idx_clics_eta_date ON clics (etablissement_id, created_at DESC);

-- Anti-fraude : un seul clic par etablissement, par jour, par visiteur.
-- Cette contrainte est le vrai rempart, pas le code applicatif.
CREATE UNIQUE INDEX idx_clics_anti_fraude
  ON clics (etablissement_id, ip_hash,
            (created_at AT TIME ZONE 'Africa/Douala')::date)
  WHERE compte = true;

-- 3. LES CONVERSATIONS ------------------------------------------------
-- Ce que le client declare avoir recu, filtre par code de reference.
-- On ne compte JAMAIS une conversation sans code : garde-fou numero 1.

CREATE TABLE conversations (
  id              BIGSERIAL PRIMARY KEY,
  etablissement_id INT NOT NULL REFERENCES etablissements(id),
  clic_id          BIGINT REFERENCES clics(id) ON DELETE SET NULL,
  ref_code         TEXT NOT NULL DEFAULT '',
  declaree_par    INT NOT NULL DEFAULT 0,      -- total annonce par le client
  date_declaration TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_conv_eta_date
  ON conversations (etablissement_id, date_declaration DESC);

-- 4. LA CHAINE DE MESURE (vue) ----------------------------------------
-- Le tableau de bord affiche cette vue, jamais un chiffre isole.
-- C'est ce qui transforme le client en controleur qualite : il voit les 4 etapes.

CREATE OR REPLACE VIEW chaine_mesure AS
SELECT
  e.id,
  e.nom,
  e.slug,
  e.ref_code,
  COALESCE(c.clics_envoyes, 0)      AS clics_envoyes,
  COALESCE(c.clics_comptes, 0)      AS clics_comptes,
  COALESCE(v.declarees, 0)          AS conversations_declarees,
```

```
  COALESCE(v.verifiees, 0)          AS conversations_verifiees,
  c.derniere_activite
FROM etablissements e
LEFT JOIN (
  SELECT etablissement_id,
         COUNT(*)                         AS clics_envoyes,
         COUNT(*) FILTER (WHERE compte)   AS clics_comptes,
         MAX(created_at)                  AS derniere_activite
  FROM clics GROUP BY etablissement_id
) c ON c.etablissement_id = e.id
LEFT JOIN (
  SELECT etablissement_id,
         MAX(declaree_par) AS declarees,
         -- Verifiee seulement si elle porte le code.
         COUNT(*) FILTER (WHERE ref_code <> '') AS verifiees
  FROM conversations GROUP BY etablissement_id
) v ON v.etablissement_id = e.id
WHERE e.actif;
```

Deux détails qui méritent une explication

Le fuseau horaire dans l'index. `AT TIME ZONE 'Africa/Douala'` fait que la journée de l'anti-fraude commence à minuit au Cameroun. Sans cela, un serveur configure en heure de Paris donnerait un décalage de plusieurs heures sur le regroupement quotidien, et le garde-fou compterait faux.

La vue fait le travail à la place du code. Compter des conversations vérifiées est une règle de gestion, pas une règle technique. En la mettant dans une vue SQL, tu n'écris pas cette logique dans trois endroits différents, et le tableau de bord ne peut pas diverger de la base.

PARTIE 3 — LE BACK-END

backend/src/db.ts

Rôle : c'est le fichier le plus important du back-end. Il contient `enregistrerClic`, la fonction qui transforme un clic en preuve horodatée. Tout le reste n'est que de l'affichage.

```
// Le serveur qui enregistre les clics. C'est le seul fichier vraiment
// important du back-end : tout le reste n'est que de l'affichage.
//
// Principe : un clic est un FAIT. Il est ecrit ici, par le serveur, horodate.
// Aucun client ne peut declarer un clic qu'il n'a pas fait.

import { Pool } from 'pg';
import crypto from 'node:crypto';

// Le sel du hachage. En production : une variable d'environnement.
// Il sert a ne jamais stocker d'IP en clair.
const SALT = process.env.CLICK_SALT ?? 'sel-de-developpement';

const pool = new Pool({
  connectionString: process.env.DATABASE_URL ??
    'postgres://plateforme:plateforme@localhost:5432/plateforme',
});

export { pool };

// Un token court et unique, qui voyage dans le lien et dans wa.me.
// 8 caracteres en base64url : illisible, mais court pour un lien WhatsApp.
function nouveauToken(): string {
  return crypto.randomBytes(6).toString('base64url');
}

// On ne veut pas l'IP en clair : personne ne doit pouvoir remonter
// a un particulier depuis la base. Le sel empeche les inversions par table.
function hacherIp(ip: string): string {
```

```
  return crypto
    .createHash('sha256')
    .update(SALT + ip)
    .digest('hex')
    .slice(0, 32);
}

export async function trouverParSlug(slug: string) {
  const { rows } = await pool.query(
    `SELECT * FROM etablissements WHERE slug = $1 AND actif`,
    [slug],
  );
  return rows[0] ?? null;
}

// Catalogue : ce que voit le visiteur. Pas de pagination
// tant qu'on n'a pas plus de 200 etablissements.
export async function listerEtablissements(categorie?: string) {
  const { rows } = await pool.query(
    `SELECT id, slug, nom, categorie, quartier, prix_affiche,
            devisee, photo_url
     FROM etablissements
     WHERE actif AND ($1::text IS NULL OR categorie = $1)
     ORDER BY quartier, nom`,
    [categorie ?? null],
  );
  return rows;
}

// LE clic. Cette fonction est le coeur du projet.
//
// Elle fait trois choses dans cet ordre :
//   1. elle cree le token et enregistre le clic ;
//   2. elle verifie si ce clic compte (anti-fraude) ;
//   3. elle renvoie l'URL WhatsApp a ouvrir.
//
// Point important : on enregistre TOUJOURS, meme un clic comptabilise comme
// frauduleux. Le journal doit etre complet pour etre auditable. C'est le champ
// `compte` qui dit si on le facture, pas la presence de la ligne.
export async function enregistrerClic(
  etablissementId: number,
  ip: string,
) {
  const token = nouveauToken();
  const ipHash = hacherIp(ip);

  // On tente l'insertion. Si l'index unique anti-fraude la refuse,
  // c'est un doublon : on garde quand meme la ligne, mais non comptabilisee.
  try {
    await pool.query(
      `INSERT INTO clics (etablissement_id, token, ip_hash)
       VALUES ($1, $2, $3)`,
      [etablissementId, token, ipHash],
    );
  } catch (err: any) {
    // violation de la contrainte unique = doublon sur la journee
    if (err?.code === '23505') {
      await pool.query(
        `INSERT INTO clics (etablissement_id, token, ip_hash,
                            compte, motif_exclu)
         VALUES ($1, $2, $3, false, 'doublon-jour')`,
        [etablissementId, token, ipHash],
      );
    } else {
      throw err;
    }
  }

  return token;
}
```

```
// La chaine affichee au client. Les 4 etapes, jamais un chiffre seul.
export async function chaineMesure(etablissementId: number) {
  const { rows } = await pool.query(
    `SELECT * FROM chaine_mesure WHERE id = $1`,
    [etablissementId],
  );
  return rows[0] ?? null;
}

// Declaration du client : combien de conversations il a recues sur SON WhatsApp.
// Rappel : cette donnee est declaree, pas verifiee.
export async function declarerConversations(
  etablissementId: number,
  nombre: number,
  refCode: string,
) {
  const { rows } = await pool.query(
    `INSERT INTO conversations (etablissement_id, declaree_par, ref_code)
     VALUES ($1, $2, $3) RETURNING *`,
    [etablissementId, nombre, refCode],
  );
  return rows[0];
}
```

Pourquoi on enregistre quand même un clic frauduleux ? Parce que le journal doit être complet pour être auditable. Si tu effaces les doublons, tu ne peux plus prouver à un client qui a triché que tu as détecté sa tentative. La ligne est conservée, le champ `compte` passe à faux, et le motif est écrit : `doublon-jour`.

Le hachage de l'IP

On ne conserve jamais une adresse IP en clair. Une IP est une donnée personnelle : elle localise une personne. Le hachage avec un sel empêche de remonter d'un haché vers l'adresse d'origine, y compris par table de correspondance. Le sel vit dans une variable d'environnement, donc il ne voyage pas dans le dépôt.

backend/src/server.ts

Rôle : les quatre routes HTTP. La deuxième est la plus importante du projet — c'est elle qui enregistre le clic et redirige vers WhatsApp.

```
// Le serveur HTTP. Quatre routes seulement :
//   GET  /api/etablissements          le catalogue
//   GET  /c/:slug                     le clic : enregistre, puis redirige
//   GET  /api/et/:id/tableau           la chaine de mesure
//   POST /api/et/:id/declaration       ce que le client declare avoir recu

import Fastify from 'fastify';
import cors from '@fastify/cors';
import {
  trouverParSlug,
  listerEtablissements,
  enregistrerClic,
  chaineMesure,
  declarerConversations,
} from './db.js';

const app = Fastify({ logger: true });

// L'API est appelee par le front Next.js, donc on autorise les appels locaux.
// En production, restreindre `origin` aux domaines reels.
app.register(cors, { origin: true });

// --- 1. Le catalogue public ------------------------------------------------
```

```
app.get('/api/etablissements', async (req) => {
  const { categorie } = req.query as { categorie?: string };
  return { etablissements: await listerEtablissements(categorie) };
});

// --- 2. LA ROUTE DU CLIC ------------------------------------------------
// C'est la route qui fait vivre le projet. Elle est volontairement separate :
// elle ne rend rien, elle enregistre et redirige. Aucun JavaScript.
//
// Le ref_code voyage dans l'URL WhatsApp pour que l'utilisateur puisse le
// renvoyer dans son message. C'est le seul moyen, depuis un numero tiers,
// d'avoir une trace que la conversation a eu lieu.

app.get('/c/:slug', async (req, reply) => {
  const { slug } = req.params as { slug: string };

  const etablissement = await trouverParSlug(slug);
  if (!etablissement) {
    return reply.code(404).send({ erreur: 'etablissement inconnu' });
  }

  const ip = req.ip;
  const token = await enregistrerClic(etablissement.id, ip);

  // WhatsApp ouvre la discussion chez le commerce, pas chez nous.
  const texte = encodeURIComponent(
    `Bonjour, je viens de votre page (ref ${etablissement.ref_code}).`,
  );
  const urlWa = `https://wa.me/${etablissement.whatsapp}?text=${texte}`;

  reply.header('Set-Cookie',
    `clic=${token}; Path=/; Max-Age=86400; SameSite=Lax`);

  // 302 : temporaire. On veut que le lien soit rejouable.
  return reply.redirect(urlWa, 302);
});

// --- 3. La chaine de mesure ---------------------------------------------

app.get('/api/et/:id/tableau', async (req) => {
  const { id } = req.params as { id: string };
  return await chaineMesure(Number(id));
});

// --- 4. La declaration du client ----------------------------------------
// Le client dit combien il a recu. On ne le croit pas : on enregistre,
// et la vue ne compte comme verifie que ce qui porte le code.

app.post('/api/et/:id/declaration', async (req, reply) => {
  const { id } = req.params as { id: string };
  const { nombre, refCode } =
    req.body as { nombre?: number; refCode?: string };

  if (typeof nombre !== 'number' || nombre < 0) {
    return reply.code(400).send({ erreur: 'nombre invalide' });
  }

  // Garde-fou : on ne compte jamais plus que les clics envoyes.
  const chaine = await chaineMesure(Number(id));
  const plafond = chaine?.clics_envoyes ?? 0;

  return reply.send({
    enregistrement: 'ok',
    valeur_annoncee: nombre,
    clics_envoyes: plafond,
    plafond_a: Math.min(nombre, plafond),
    remarque:
      nombre > plafond
        ? 'Valeur superieure au nombre de clics envoyes : impossible.'
        : null,
```

```
  });
});

const port = Number(process.env.PORT ?? 3001);
await app.listen({ port, host: '0.0.0.0' });
```

Trois détails qui font la différence

Aucun JavaScript sur la route du clic. La route `/c/:slug` enregistre puis renvoie une redirection HTTP 302. Un clic mesuré en JavaScript peut être bloqué par un bloqueur de publicités, par une extension, ou par un navigateur qui l'interprète mal. Une redirection serveur ne peut pas échouer.

Le statut 302, pas 301. Une redirection permanente serait mémorisée par le navigateur et le lien ne fonctionnerait qu'une seule fois. Ici le lien reste rejouable, ce qui est nécessaire quand un utilisateur revient sur la fiche.

Le code de référence voyage dans le message. Le message pré-rempli contient `ref EST-042-A7`. C'est la seule trace possible depuis un numéro tiers : si l'utilisateur renvoie ce code dans son message, on sait que la conversation a eu lieu. Ce n'est pas une preuve, c'est un filtre — et il suffit à rendre la fraude inutile quand on ne paie jamais à l'unité.

backend/src/seed.ts

Rôle : douze établissements de Douala avec leurs vrais quartiers, pour que la page d'accueil ait de quoi ressembler à un produit dès le premier jour.

```
// Donnees de depart. Douze etablissements realistes, avec leurs quartiers.
// But : que la page d'accueil ait de quoi ressembler a un produit, le jour 1.

import { pool } from './db.js';

const etablissements = [
  // Restaurants
  ['restaurant-le-bonamoussadi', 'Le Bonamoussadi', 'restaurant',
   'Bonamoussadi', 4500, 'Plat du jour (ndole)', '237691234501'],
  ['restaurant-akwa-grill', 'Akwa Grill', 'restaurant',
   'Akwa', 6500, 'Grillade complete', '237691234502'],
  ['restaurant-bonapriso-terrasse', 'La Terrasse Bonapriso', 'restaurant',
   'Bonapriso', 9000, 'Diner pour deux', '237691234503'],
  ['restaurant-makepe-poulet', 'Le Poulet de Makepe', 'restaurant',
   'Makepe', 3000, 'Poulet braise', '237691234504'],
  ['restaurant-bali-bonapriso', 'Bali Bonapriso', 'restaurant',
   'Bonapriso', 5500, 'Pizza familiale', '237691234505'],
  ['restaurant-logbaba-cuisine', 'Cuisine Logbaba', 'restaurant',
   'Logbaba', 2500, 'Soupe et beignets', '237691234506'],

  // Meubles
  ['meuble-studio-akwa', 'Studio Akwa Centre', 'meuble',
   'Akwa', 25000, 'par nuit', '237691234601'],
  ['meuble-familial-bonamoussadi', 'Familial Bonamoussadi', 'meuble',
   'Bonamoussadi', 45000, 'par nuit', '237691234602'],
  ['meuble-lodge-makepe', 'Lodge Makepe', 'meuble',
   'Makepe', 35000, 'par nuit', '237691234603'],
  ['meuble-residence-bonapriso', 'Residence Bonapriso', 'meuble',
   'Bonapriso', 60000, 'par nuit', '237691234604'],
  ['meuble-studio-logbaba', 'Studio Logbaba', 'meuble',
   'Logbaba', 15000, 'par nuit', '237691234605'],
  ['meuble-villa-bali', 'Villa Bali', 'meuble',
   'Bali', 90000, 'par nuit', '237691234606'],
];

// Le ref_code est ce que l'utilisateur renvoie dans son message WhatsApp.
// Format : prefixe - numero - suffixe. Court, illisible, unique.
async function main() {
```

```
  for (const [slug, nom, categorie, quartier, prix, devisee, whatsapp]
       of etablissements) {
    const refCode =
      `${String(slug).toUpperCase().slice(0, 3)}-` +
      `${Math.floor(Math.random() * 900 + 100)}-` +
      `${Math.random().toString(36).slice(2, 5).toUpperCase()}`;

    await pool.query(
      `INSERT INTO etablissements
         (slug, nom, categorie, quartier, prix_affiche,
          devisee, whatsapp, ref_code)
       VALUES ($1,$2,$3,$4,$5,$6,$7,$8)
       ON CONFLICT (slug) DO NOTHING`,
      [slug, nom, categorie, quartier, prix, devisee, whatsapp, refCode],
    );
  }

  const { rows } = await pool.query(
    `SELECT COUNT(*)::int AS total FROM etablissements`,
  );
  console.log(`Seed termine : ${rows[0].total} etablissements en base.`);
  await pool.end();
}

main().catch((err) => {
  console.error(err);
  process.exit(1);
});
```

PARTIE 4 — LE FRONT-END

frontend/app/layout.tsx

Rôle : la racine du site. Le titre et la description sont ici, parce qu'ils sont ce que Google lit en premier.

```
// Racine : tout est en Server Component.
// Aucune feuille de style externe pour le premier test : on veut voir
// le produit avant de discuter de son apparence.

export const metadata = {
  title: 'Douala — a manger, a dormir',
  description:
    'Restaurants et meubles a Douala. Prix affiche avant le clic, ' +
    'informations verifiees.',
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="fr">
      <body>{children}</body>
    </html>
  );
}
```

frontend/app/page.tsx

Rôle : le catalogue. Sa seule responsabilité est d'afficher le prix avant le clic — c'est l'avantage compétitif du projet.

```
// La page d'accueil : le catalogue.
//
```

```
// Elle fait une chose simple et elle la fait bien : afficher le prix AVANT
// le clic. C'est notre seul vrai avantage sur une annonce Facebook.
//
// Note sur le rendu : tout est en HTML cote serveur (Server Component).
// Pas de JavaScript pour afficher une carte. C'est ce qui rend la page
// indexable par Google, donc ce qui rend le SEO (trajectoire T1) possible.

import Link from 'next/link';

const API = process.env.API_URL ?? 'http://localhost:3001';

export type Etablissement = {
  id: number;
  slug: string;
  nom: string;
  categorie: 'restaurant' | 'meuble';
  quartier: string;
  prix_affiche: number;
  devisee: string;
  photo_url: string | null;
};

async function getCatalogue(
  categorie?: string,
): Promise<Etablissement[]> {
  const url =
    categorie && categorie !== 'tout'
      ? `${API}/api/etablissements?categorie=${categorie}`
      : `${API}/api/etablissements`;

  // revalidate : le catalogue change peu, on ne veut pas requeter l'API
  // a chaque visite. 60 secondes suffisent.
  const res = await fetch(url, { next: { revalidate: 60 } });
  if (!res.ok) return [];
  const data = await res.json();
  return data.etablissements ?? [];
}

// Prix format a la francaise : espace insecable, sans decimale.
function fcfa(n: number): string {
  return `${n.toLocaleString('fr-FR').replace(/\u202f|\u00a0/g, ' ')} FCFA`;
}

export default async function Accueil({
  searchParams,
}: {
  searchParams: Promise<{ categorie?: string }>;
}) {
  const { categorie = 'tout' } = await searchParams;
  const etablissements = await getCatalogue(categorie);

  return (
    <main>
      <header>
        <h1>Ce qui se mange, ce qui se dort, a Douala</h1>
        <p>Prix affiche avant de cliquer. Verification a la main.</p>
      </header>

      {/* Filtres : liens simples, pas de client state.
          Ca reste utilisable meme si le JavaScript ne charge pas. */}
      <nav aria-label="Filtres">
        {[
          ['tout', 'Tout'],
          ['restaurant', 'Restaurants'],
          ['meuble', 'Meubles'],
        ].map(([valeur, label]) => (
          <Link
            key={valeur}
            href={valeur === 'tout' ? '/' : `/?categorie=${valeur}`}
            aria-current={categorie === valeur ? 'page' : undefined}
          >
```

```
            {label}
          </Link>
        ))}
      </nav>

      <section>
        {etablissements.length === 0 && (
          <p>Aucun etablissement. Lancez le back-end puis le seed.</p>
        )}

        <ul>
          {etablissements.map((e) => (
            <li key={e.slug}>
              <article>
                <h2>
                  {/* Toute la carte est un lien : le clic compte sur la fiche,
                      pas sur un bouton isole. */}
                  <Link href={`/e/${e.slug}`}>{e.nom}</Link>
                </h2>

                <p className="quartier">{e.quartier}</p>

                {/* LE prix, avant le clic. C'est l'argument. */}
                <p className="prix">
                  <strong>{fcfa(e.prix_affiche)}</strong>
                  <span>{e.devisee}</span>
                </p>

                <Link className="cta" href={`/e/${e.slug}`}>
                  Decouvrir plus
                </Link>
              </article>
            </li>
          ))}
        </ul>
      </section>
    </main>
  );
}
```

Deux décisions valent une explication. D'abord `next: { revalidate: 60 }` : le catalogue bouge peu, et re-requêter l'API à chaque visite poserait un problème de latence inutile. Ensuite les filtres sont des `Link` de Next, pas un état côté client : la page reste utilisable même si le JavaScript ne se charge pas, et chaque filtre a sa propre URL, donc indexable.

`frontend/app/e/[slug]/page.tsx`

Rôle : la fiche d'un établissement, et surtout le bouton. Le bouton ne pointe pas vers WhatsApp — il pointe vers notre route `/c/:slug`. C'est le point le plus important du front-end.

```
// La fiche d'un etablissement.
//
// Le bouton « Decouvrir plus » ne pointe PAS vers WhatsApp directement.
// Il pointe vers notre route /c/:slug, qui enregistre le clic (serveur)
// et redirige ensuite vers WhatsApp.
//
// C'est le point le plus important de tout le front-end.
// Si on mettait wa.me directement dans le href, on n'aurait aucune mesure.
// Un clic non mesure est un clic invendable.

import Link from 'next/link';
import { notFound } from 'next/navigation';

const API = process.env.API_URL ?? 'http://localhost:3001';

type Etablissement = {
  id: number;
```

```
  slug: string;
  nom: string;
  categorie: string;
  quartier: string;
  prix_affiche: number;
  devisee: string;
  photo_url: string | null;
  whatsapp: string;
  ref_code: string;
};

function fcfa(n: number): string {
  return `${n.toLocaleString('fr-FR').replace(/\u202f|\u00a0/g, ' ')} FCFA`;
}

export default async function Fiche({
  params,
}: {
  params: Promise<{ slug: string }>;
}) {
  const { slug } = await params;

  const res = await fetch(`${API}/api/etablissements`, {
    next: { revalidate: 60 },
  });
  const tous: Etablissement[] = (await res.json()).etablissements ?? [];
  const etablissement = tous.find((e) => e.slug === slug);

  if (!etablissement) notFound();

  return (
    <main>
      <article>
        <h1>{etablissement.nom}</h1>
        <p className="quartier">{etablissement.quartier}</p>

        <p className="prix">
          <strong>{fcfa(etablissement.prix_affiche)}</strong>
          <span>{etablissement.devisee}</span>
        </p>

        {/* LE point central : on passe par /c/:slug, pas par wa.me. */}
        <a className="cta"
           href={`http://localhost:3001/c/${etablissement.slug}`}>
          Decouvrir plus sur WhatsApp
        </a>

        {/* Transparence : le client sait que son clic sera compte.
            La transparence est une contre-mesure anti-fraude. */}
        <p className="mention">
          Votre clic est enregistre. Vous engagez la conversation
          directement avec l'etablissement, sur son numero WhatsApp.
        </p>
      </article>

      <Link href="/">Retour au catalogue</Link>
    </main>
  );
}
```

> **Le piège à éviter absolument** : mettre `https://wa.me/23769...` directement dans le `href`. Le clic part, WhatsApp s'ouvre, et il n'existe aucune trace côté serveur. Un clic non mesuré est un clic invendable : tu ne peux ni le facturer, ni prouver au client qu'il a eu lieu.

frontend/app/dashboard/page.tsx

Rôle : le tableau de bord du client. Sa règle de conception est qu'il affiche la chaîne complète des quatre étapes, jamais un chiffre isolé.

```
// Le tableau de bord du client.
//
// La regle de conception ici : on affiche la CHAINE, jamais un chiffre isole.
//   clics envoyes -> clics comptes -> conversations declarees -> verifiees
//
// Pourquoi : c'est ce qui transforme le client en controleur qualite.
// Il voit la meme donnee que nous. Si l'ecart entre les 4 etapes est
// visible, c'est lui qui le remarque. On n'a plus a le suspecter.

const API = process.env.API_URL ?? 'http://localhost:3001';

type Chaine = {
  id: number;
  nom: string;
  slug: string;
  ref_code: string;
  clics_envoyes: number;
  clics_comptes: number;
  conversations_declarees: number;
  conversations_verifiees: number;
  derniere_activite: string | null;
};

export default async function TableauDeBord() {
  // En production : l'identifiant vient de la session du client.
  // Ici on prend le premier etablissement, pour la demonstration.
  const res = await fetch(`${API}/api/etablissements`, {
    cache: 'no-store',
  });
  const liste: { id: number }[] =
    (await res.json()).etablissements ?? [];
  if (liste.length === 0) {
    return <main><p>Aucun etablissement en base.</p></main>;
  }

  const etablissementId = liste[0].id;

  const tRes = await fetch(`${API}/api/et/${etablissementId}/tableau`, {
    cache: 'no-store',
  });
  const chaine: Chaine = await tRes.json();

  // Le taux de verification est l'indicateur honnete : il dit combien
  // des conversations declarees portent vraiment le code de reference.
  const tauxVerification =
    chaine.conversations_declarees > 0
      ? Math.round(
          (chaine.conversations_verifiees /
            chaine.conversations_declarees) * 100,
        )
      : 0;

  return (
    <main>
      <h1>Tableau de bord — {chaine.nom}</h1>

      <section>
        <h2>Votre chaine de mesure</h2>
        <table>
          <tbody>
            <tr>
              <th>Clics envoyes vers votre WhatsApp</th>
              <td>{chaine.clics_envoyes}</td>
            </tr>
            <tr>
              <th>Clics retenus dans le decompte</th>
              <td>{chaine.clics_comptes}</td>
```

```
            </tr>
            <tr>
              <th>Conversations declarees</th>
              <td>{chaine.conversations_declarees}</td>
            </tr>
            <tr>
              <th>Conversations verifiees (avec code)</th>
              <td>{chaine.conversations_verifiees}</td>
            </tr>
          </tbody>
        </table>
      </section>

      <section>
        <h2>Ce que cela signifie</h2>
        <ul>
          <li>
            Les <strong>clics</strong> sont comptes par notre serveur.
            Ils sont fiables : votre facture ne repose pas dessus
            uniquement.
          </li>
          <li>
            Les <strong>conversations declarees</strong> viennent de
            votre WhatsApp. Nous ne pouvons pas les verifier.
          </li>
          <li>
            Les <strong>conversations verifiees</strong> sont celles
            ou l'utilisateur a renvoye le code
            <code>{chaine.ref_code}</code> dans son message.
          </li>
        </ul>
        <p>
          Taux de verification actuel : <strong>{tauxVerification} %</strong>.
          Si ce taux est bas, demandez a vos clients de renvoyer le code.
        </p>
      </section>

      <section>
        <h2>Comment vous etes facture</h2>
        <p>
          Par <strong>abonnement mensuel de presence</strong>.
          Vous etes visible sur le site tant que l'abonnement est actif.
        </p>
        <p>
          Ce qui compte pour vous n'est pas le nombre de clics, mais le
          nombre de conversations qui aboutissent. Les clics dependent de
          notre trafic, que nous controlons peu. Votre visibilite, elle,
          ne depend que de nous.
        </p>
      </section>
    </main>
  );
}
```

Le taux de vérification affiché en bas du tableau est l'indicateur honnête du système : il dit combien des conversations déclarées portent réellement le code de référence. S'il reste bas, la parade n'est pas technique, elle est commerciale : demander aux clients de renvoyer le code dans leurs messages WhatsApp.

`docker-compose.yml`

Rôle : la base de données, en une commande. Le fuseau horaire est forcé sur `Africa/Douala` pour que le regroupement quotidien de l'anti-fraude soit juste.

```
services:
  db:
    image: postgres:18-alpine
```

```
    container_name: plateforme-db
    environment:
      POSTGRES_USER: plateforme
      POSTGRES_PASSWORD: plateforme
      POSTGRES_DB: plateforme
      # Africa/Douala : le decoupage par jour de l'index anti-fraude
      # doit suivre l'heure locale du Cameroun, pas celle du serveur.
      TZ: Africa/Douala
      PGTZ: Africa/Douala
    ports:
      - '5432:5432'
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U plateforme']
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  db_data:
```

PARTIE 5 — MONTAGE PAS À PAS

5. Lancer l'application en une commande

Un script est fourni à la racine du dossier : `./demarrer.sh`. Il vérifie les prérequis, démarre la base, crée le schéma seulement s'il est absent, insère les données, puis lance le back-end et le front-end en arrière-plan.

```
# Tout lancer
./demarrer.sh

# Tout arreter
./demarrer.sh arret

# Suivre les journaux
tail -f /tmp/back.log
tail -f /tmp/front.log
```

Le script est volontairement idempotent : le relancer ne réinitialise pas la base et ne duplique pas les établissements. C'est utile pendant une journée de développement où on relance souvent.

> **L'erreur la plus probable au premier lancement** est que Docker soit installé mais arrêté. Le script la détecte et affiche la commande selon ton système :
>
> ```
> Sur Linux    : sudo systemctl start docker
> Sur Mac      : ouvre Docker Desktop, attends le logo
> Sur Windows  : ouvre Docker Desktop, attends le logo
>
> Pour verifier :  docker info
> ```
>
> Sans ce démon, rien ne démarre, et aucune base de données n'est disponible.

6. Les prérequis

- Node.js 20 ou plus — vérifier avec `node -v`
- Docker — vérifier avec `docker -v`

- Rien d'autre. Pas de Postgres à installer : Docker s'en charge.

7. Les commandes, une par une

```
# 1. Demarrer la base
cd plateforme
docker compose up -d
docker compose ps          # doit afficher db en healthy

# 2. Creer le schema
cd backend
export DATABASE_URL="postgres://plateforme:plateforme@localhost:5432/plateforme"
npm install
npm run initdb

# 3. Mettre des donnees de depart (12 etablissements)
npm run seed

# 4. Demarrer le back-end  (terminal 1)
npm run dev                # http://localhost:3001
```

```
# 5. Demarrer le front-end  (terminal 2)
cd frontend
npm install
npm run dev                # http://localhost:3000

# 6. Arreter
docker compose down        # garde les donnees
docker compose down -v     # efface aussi la base
```

PARTIE 6 — VÉRIFIER QUE ÇA MARCHE

8. Voir le produit

Page	Adresse	Ce qu'on doit voir
Catalogue	`localhost:3000`	Les 12 établissements, avec le prix affiché
Fiche	`localhost:3000/e/restaurant-le-bonamoussadi`	Le prix et le bouton « Decouvrir plus »
Tableau de bord	`localhost:3000/dashboard`	Les 4 lignes de la chaîne de mesure

9. Vérifier la mesure de clic

C'est le test le plus important du projet. Depuis un terminal :

```
curl -sI http://localhost:3001/c/restaurant-le-bonamoussadi | head -5
```

Tu dois voir une redirection `302` vers `wa.me`. Si tu vois une page blanche ou une erreur 404, l'établissement n'a pas été chargé : relance le seed.

Puis regarde ce qui a été réellement écrit en base :

```
docker compose exec db psql -U plateforme -d plateforme \
```

```
  -c "SELECT token, compte, motif_exclu FROM clics ORDER BY id DESC LIMIT 5;"
```

10. Tester l'anti-fraude

Relance la commande du point 8 deux fois de suite, sans changer d'adresse IP.

Résultat attendu :

Ligne	compte	motif_exclu	Explication
1	`t`	*null*	Le premier clic de la journée compte
2	`f`	`doublon-jour`	Refusé par l'index unique
3	`f`	`doublon-jour`	Idem

C'est le cœur de la démonstration : la base elle-même a refusé la seconde ligne, sans que ce soit le code applicatif qui décide.

11. État de vérification de ce code

Ce qui a été vérifié : les deux projets compilent sans erreur TypeScript (`tsc --noEmit` passe sur le back-end et le front-end), le serveur Fastify démarre et répond, le script `demarrer.sh` a une syntaxe bash valide, et le fichier `docker-compose.yml` est accepté par `docker compose config`.

Trois défauts ont été trouvés et corrigés pendant cette vérification : le chemin du schéma était faux dans le script `initdb`, `next-env.d.ts` manquait, et l'API catalogue ne renvoyait pas d'identifiant, ce qui cassait le tableau de bord.

Ce qui n'a pas pu être vérifié : l'exécution complète avec la base de données. Le démon Docker ne tourne pas sur la machine où ce code a été écrit, et il exige les privilèges root pour démarrer. Les commandes des sections 5 et 7 n'ont donc pas été exécutées de bout en bout : elles restent à confirmer chez toi.

Concrètement, ce qui reste non testé : la création effective des tables, le `seed`, la redirection 302 vers `wa.me`, et le refus du doublon par l'index unique. Le code de ces chemins est relu et typé, mais pas exécuté.

12. Ce qui manque volontairement

Ce PDF contient le strict minimum qui rend le tracker fonctionnel. Ces éléments viennent au palier suivant, pas avant :

Élément	Pourquoi plus tard
Session et authentification du client	Aucune donnée à protéger tant qu'il n'y a pas d'abonnement réel
Redis	Limitation de débit et cache : inutile avec 12 établissements
Facturation et abonnements	Le prix n'est pas tranché. Ne pas coder avant de décider
Upload de photos et `sharp`	Pas de photos pour l'instant ; les URLs sont vides
API WhatsApp Cloud et webhooks	Nécessite un numéro et un compte Meta validé
Mobile money	On ne touche pas à l'argent. C'est un choix, pas un oubli
Tailwind et le design	Le produit d'abord, l'apparence ensuite

Le tableau ci-dessus n'est pas une liste de tâches oubliées. Chaque ligne est un choix : le projet avance sur ce qui est nécessaire pour valider l'hypothèse des 20 établissements, et rien d'autre ne justifie encore d'être écrit.

13. La règle de séquencement

> Ces cinq éléments de base peuvent démarrer maintenant, sans attendre la validation du prix de vente.
>
> En revanche, **aucun travail de trafic** — SEO, Instagram, TikTok, groupes WhatsApp — ne commence avant que les établissements soient signés. Construire du trafic avant de savoir ce qu'on vend, c'est la manière la plus fiable de perdre de l'argent.